Dev Mobile Android

Android Views

Olivier DENIER - v260208

Plan

- Introduction
- Le design
- Les ViewGroups : FrameLayout, LinearLayout, ConstraintLayout
- Les Views : Button, TextView, EditText, ImageView, …
- Les listes : RecyclerView
- Autres composants : Fenêtres Dialog, Toast et SnackBar, Menus, Pickers

Introduction

Android Views ?

Il s’agit de l’ensemble des composants graphiques utilisés pour décrire/dessiner les écrans des applications Android.

Les écrans seront définis dans des fichiers XML : les layouts
Le XML ne sert qu’à définir les écrans, mais à l’exécution, les éléments XML sont “traduits” en composants graphiques (objets Java).

Référence : https://developer.android.com/develop/ui/views/layout/declaring-layout?hl=fr

Historique

Les Android Views = Système historique de design des écrans des apps Android

Mais de nos jours, c'est Jetpack Compose qui est préconisé.

Jetpack Compose ?
Ensemble de composants graphiques permettant de créer des écrans de manière déclarative en Kotlin (pas en XML).
La v1 date de Juillet 2021
🫣 Nous n'étudierons pas Jetpack Compose dans ce cours.

Référence : https://developer.android.com/compose

Le Design

Les Layouts

Les **layouts** (fichiers XML) décrivent le design des écrans et sont stockés dans le répertoire “res/layout”.

Un layout est un assemblage de composants graphiques permettant de décrire le design :

- d’un écran,
- d’une partie d’écran ou
- d’un composant graphique.

Référence : https://developer.android.com/guide/topics/resources/layout-resource

Les composants du Design

Deux types de composants de base sont utiles pour définir l'interface graphique (le design des écrans) de notre application :

- Les ViewGroups
  Composants "containers" permettant de contenir plusieurs composants : des ViewGroups et des Views.
- Les Views
  Composants "simples" comme un bouton, un label, un champ de saisie, un composant pour afficher une image, …

Android Views XML

La définition/description des écrans avec les **Android Views** est faite en XML.
Pour chaque composant graphique, une balise (tag) XML lui est associée.

Par exemple :

- <Button … /> pour décrire un bouton
- <TextView … /> pour décrire un label
- <EditText … /> pour décrire un champ de saisie
- <ImageView … /> pour décrire un composant affichant une image
- …

Chaque composant graphique = 1 tag XML = 1 classe Java/Kotlin

Android Views XML : attributs (1/3)

Au sein des balises (tags) et selon les composants, des attributs spécifiques permettant de définir les caractéristiques du composant.

Voici quelques attributs communs à tous les composants :

- android:id : définit l'ID (nom unique) du composant
- android:layout_width : définit la largeur du composant
- android:layout_height : définit la hauteur du composant

⚠️ Chaque composant (ViewGroup ou View) doit OBLIGATOIREMENT définir un android:layout_width et android:layout_height

Android Views XML : attributs (2/3)

Voici quelques attributs très courants :

- android:layout_marginXXX : définit les marges extérieures du composant
- android:paddingXXX : définit les marges intérieures du composant
- android:gravity : définit le positionnement du contenu du composant
- android:layout_gravity : définit le positionnement du composant
- android:background : définit la couleur de fond du composant
- android:visibility : définit si le composant est visible ou non
- android:hint : définit un “placeholder” dans le champ de saisie du composant

Android Views XML : attributs (3/3)

Voici quelques attributs très courants :

- android:text : définit le texte du champ de saisie du composant
- android:textColor : définit la couleur du texte du composant
- android:textSize : définit la taille du texte du composant
- android:textStyle : définit le style du texte du composant
- android:maxLines : définit le nombre de ligne pour le texte
- android:enabled : définit si le composant est cliquable ou non
- style : définit le style du composant

Remarque : les attributs sont généralement préfixés de android: ou app:

Layout et Views

Pour définir le layout d'une Activity (son design) :
setContentView(R.layout.activity_main)
R.layout.activity_main correspond au fichier "res/layout/activity_main.xml"

Pour pouvoir utiliser les composants graphiques (Views) au sein du code, il va falloir leurs donner un **id** dans le XML et les récupérer dans le code :
val myButton = findViewById(R.id.myButton)
R.id.myButton correspond à l'ID définit fichier "res/layout/activity_main.xml"

⚠️ Attention au nommage des layouts, des ids, …

Les ViewGroups

Viewgroups ?

Les **Viewgroups** sont des composants containers qui souvent n'ont pas de représentation graphique mais servent à positionner les autres composants graphiques pour composer une interface graphique.

On peut lister par exemple :

- FrameLayout
- LinearLayout
- ConstraintLayout (remplace le RelativeLayout)
- TableLayout

Référence : https://developer.android.com/develop/ui/views/layout/declaring-layout?hl=fr

FrameLayout

Le FrameLayout permet de positionner tous ses composants enfants en haut à gauche (effet de superposition) de celui-ci.

Référence : https://developer.android.com/reference/android/widget/FrameLayout

LinearLayout

Le LinearLayout permet de positionner tous ses composants enfants

- de gauche à droite : android:orientation="horizontal"
  ou
- de haut en bas : android:orientation="vertical"

Attribut intéressant pour les composants définis dans un LinearLayout :

- android:layout_weight : définit un poids au composant (équivalent d'un pourcentage)

Références :
https://developer.android.com/develop/ui/views/layout/linear
https://developer.android.com/reference/android/widget/LinearLayout

ConstraintLayout (1/4)

Ce type de layout permet de positionner les composants qu'il contient les uns par rapport aux autres.

Le composant androidx.constraintlayout.widget.ConstraintLayout remplace le RelativeLayout.

Pour l'utiliser, il est nécessaire d'importer la librairie qui le contient :
implementation("androidx.constraintlayout:constraintlayout:2.2.1")

Références : https://developer.android.com/jetpack/androidx/releases/constraintlayout#kts
https://developer.android.com/develop/ui/views/layout/constraint-layout

ConstraintLayout (2/4)

Quelques avantages du ConstraintLayout :

- Temps d'affichage plus performant (moins d'imbrication entre les composants)
- Pas besoin de respecter un ordre dans les composants

⚠️ Par contre les composants doivent avoir un android:id défini

⚠️ Les composants au sein d'un ConstraintLayout doivent **obligatoirement** définir le positionnement de 2 à 4 de leurs bords/côtés :

- app:layout_constraintTop_toTopOf/toBottomOf : bord du haut
- app:layout_constraintStart_toStartOf/toEndOf : bord gauche (affichage de gauche à droite)
- app:layout_constraintEnd_toEndOf/toStartOf : bord droit (affichage de gauche à droite)
- app:layout_constraintBottom_toBottomOf/toTopOf : bord du bas

ConstraintLayout (3/4)

D’autres attributs du ConstraintLayout :

- app:layout_constraintWidth / _min / _max / _default / _percent
- app:layout_constraintHeight / _min / _max / _default / _percent
- app:layout_constraintHorizontal_weight / Vertical_weight
- app:layout_constraintHorizontal_bias / Vertical_bias
- app:layout_constraintHorizontal_chainStyle / Vertical_chainStyle
- app:layout_constraintGuide_begin / _end / _percent
- app:layout_constraintBaselineXXX
- app:layout_constraintCircleXXX
- ~~app:layout_constraintLeftXXX~~ à remplacer par constraintStartXXX
- ~~app:layout_constraintRightXXX~~ à remplacer par constraintEndXXX

ConstraintLayout (4/4)

Pour faciliter l'utilisation des ConstraintLayout, il est possible d'utiliser :

- Des guidelines : ligne invisible verticale ou horizontale à positionner (dp ou %) dans l'écran pour définir l'alignement des composants
  <androidx.constraintlayout.widget.Guideline
  Attributs utiles : android:id, app:layout_constraintGuide_begin|end|percent et android:orientation

- Des barriers : ligne invisible verticale ou horizontale dans l'écran pour définir l'alignement de tous les composants liés à cette barrier
  <androidx.constraintlayout.widget.Barrier
  Attributs utiles : app:barrierDirection et app:constraint_referenced_ids

Autres types de layouts

D’autres anciens layouts mais obsolètes :

- AbsoluteLayout : NE PAS utiliser
- GridLayout : Déconseillé
- TableLayout : Déconseillé
- RelativeLayout : Déconseillé => utiliser le ConstraintLayout à la place

Attributs obligatoires

Tous les composants doivent obligatoirement contenir les 2 attributs :

- android:layout_width=”XXX”
- android:layout_height="XXX"

La valeur “XXX” peut être :

- match_parent : le composant prend toute la place disponible
- wrap_content : le composant ne prend que la place qui lui est nécessaire
- une valeur numérique (dp) : le composant prend la dimension définie

Unité dp (ou dip) = Density Independent Pixels

Les Views

View ?

Il s’agit des composants graphiques visibles tels que :

- les boutons,
- les composants pour afficher des libellés et du texte,
- les composants pour saisir du texte,
- les composants pour afficher des images,
- les cases à cocher,
- les boutons radios,
- les barres de progression,
- …

Référence : https://developer.android.com/reference/android/view/View

Button

Il s’agit de bouton.

Elément XML : <Button … />

Classe Java : android.widget.Button

Attributs utiles :

android:text : Définit le texte à afficher dans le bouton

~~android:onClick~~ : Définit la méthode à appeler au clic (à ne plus utiliser)

android:enabled : Définit si le bouton est cliquable ou pas

Référence : https://developer.android.com/reference/android/widget/Button

TextView (Libellé et texte)

Il s’agit de composant pour afficher des libellés et du texte.

Elément XML : <TextView … />

Classe Java : android.widget.TextView

Attributs utiles :
android:text : Définit le texte à afficher
android:labelFor : Définit si le composant est le label d’un champ de saisie
android:ellipsize : Définit si le texte peut être tronqué
android:drawableXXX : Définit une image à afficher à côté du texte
Android:enabled : Définit si le texte est cliquable ou pas
android:maxLines : Définit le nombre de ligne à afficher

Variante : AutoCompleteTextView, CheckedTextView

Référence : https://developer.android.com/reference/android/widget/TextView

EditText (champ de saisie)

Il s’agit de composant pour saisir une information.

Elément XML : <EditText … />

Classe Java : android.widget.EditText

Attributs utiles :
android:text : Définit le texte à afficher
android:ellipsize : Définit si le texte peut être tronqué
android:drawableXXX : Définit une image à afficher à côté du texte
android:enabled : Définit si le texte est cliquable ou pas
android:inputType : Définit le type de la donnée à saisir
android:password : Définit si le champ est pour un mot de passe
android:hint : Définit un “placeholder”
android:lines : Définit si le nombre de lignes visibles du composant
android:maxLines : Définit si le nombre de lignes max du composant (“1” = champ de saisie simple)
android:maxLength : Définit le nombre max de caractères saisissables

Référence : https://developer.android.com/reference/android/widget/EditText

Image

Il s’agit de composant pour afficher des images.

Elément XML : <ImageView … />

Classe Java : android.widget.ImageView

Attributs utiles :
android:src : Définit l’image à afficher
android:contentDescription : Définit un texte descriptif de l’image (accessibilité)

Variante : ImageButton

Référence : https://developer.android.com/reference/android/widget/ImageView

Checkbox

Il s’agit de composant pour afficher des cases à cocher.

Elément XML : <CheckBox … />

Classe Java : android.widget.CheckBox

Attributs utiles :
android:checked : Définit si la case à cocher est cochée ou pas

Référence : https://developer.android.com/reference/android/widget/CheckBox

RadioButton et RadioGroup

Il s’agit de composant pour afficher des boutons radios.

Elément XML : <RadioGroup …/> et <RadioButton … />

Classe Java : android.widget.RadioGroup et android.widget.RadioButton

Attributs utiles :
android:checked : Définit si le bouton radio est coché ou pas

Remarque : Pour faire en sorte que les RadioButton soient exclusifs, ils faut les définir au sein d’un RadioGroup

Référence : https://developer.android.com/reference/android/widget/RadioButton

ProgressBar

Il s’agit de composant pour afficher une barre de progression.

Elément XML : <ProgressBar … />

Classe Java : android.widget.ProgressBar

<u>Attributs utiles :</u>
android:indeterminate : Définit si la progression est déterminée ou pas
android:progress : Définit le niveau de progression
android:min : Définit le niveau minimal de la progression
android:max : Définit le niveau maximal de la progression

SeekBar

Il s’agit d’un composant pour afficher une jauge (barre horizontale avec un curseur).

Elément XML : <SeekBar … />

Classe Java : android.widget.SeekBar

Attributs utiles :
android:indeterminate : Définit si la progression est déterminée ou pas
android:progress : Définit le niveau de progression
android:min : Définit le niveau minimal de la progression
android:max : Définit le niveau maximal de la progression
android:thumb : Définit le design du curseur

Référence : https://developer.android.com/reference/android/widget/SeekBar

ScrollView et HorizontalScrollView

Il s’agit d’un composant container permettant d’afficher une barre de scroll (verticale ou horizontale) pour afficher un ensemble de composants qui ne sont pas tous affichables à l’écran.

Elément XML : <ScrollView … />
Elément XML : <HorizontalScrollView … />

Classe Java : android.widget.ScrollView

⚠️ Ces composants ne peuvent contenir qu’un seul composant enfant

Référence : https://developer.android.com/reference/android/widget/ScrollView

Switch et ToggleButton

Il s’agit de composant pour afficher un interrupteur.

Elément XML : <Switch … />
Elément XML : <ToggleButton … />

Classe Java : android.widget.Switch
Classe Java : android.widget.ToggleButton

Attributs utiles :
android:textOff : Définit le texte à afficher à l’état OFF
android:textOn : Définit le texte à afficher à l’état ON

Référence : https://developer.android.com/reference/android/widget/ToggleButton
https://developer.android.com/reference/android/widget/Switch

Space

Il s’agit d’un composant pour définir un espace vide.

Elément XML : <Space … />

Classe Java : android.widget.Space

Référence : https://developer.android.com/reference/android/widget/Space

Spinner

Il s’agit d’un composant pour afficher une liste déroulante.

Elément XML : <Spinner … />

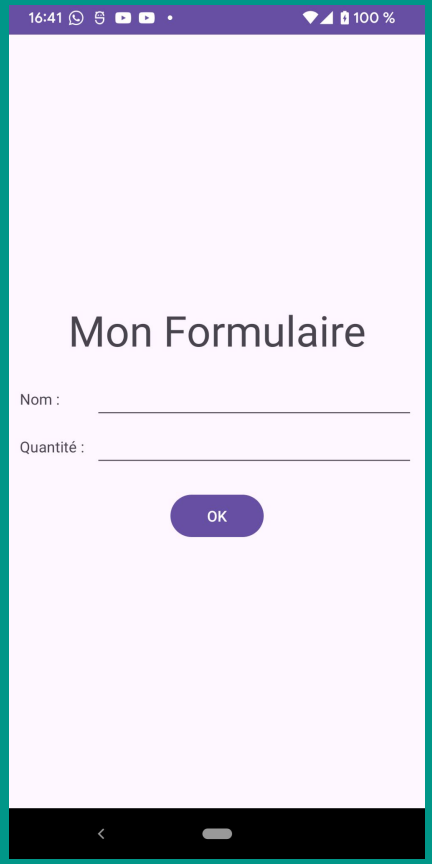
Classe Java : android.widget.Spinner

Référence : https://developer.android.com/reference/android/widget/Spinner

TD-MyDesign

Exercice 1 :

1. Dans Android Studio, créez un nouveau projet Android “MyDesign”
2. Implémentez une Activity (comme ci-contre) avec un LinearLayout comportant
   a. Un libellé “Nom” et son champ de saisie textuel
   b. Un libellé “Quantité” et son champ de saisie numérique
   c. Un bouton “OK”



Exercice 2 :

1. Faites un deuxième layout identique (même design) au premier mais en remplaçant le LinearLayout par un ConstraintLayout

Les listes

Composants pour les listes

Plusieurs composants sont possibles pour afficher des listes d'éléments :

- ListView : remplacé par le RecyclerView
- GridView : remplacé par le RecyclerView
- RecyclerView

Le principe est d'avoir un design unique (même si ce n'est pas obligatoire) par ligne pour réutiliser les composants graphiques pour afficher les informations et permettre ainsi un scroll très fluide.

RecyclerView

Le RecyclerView est le composant graphique permettant d'afficher une liste (ou grille) scrollable d'informations.

Un RecyclerView a besoin d'un Adapter pour fonctionner.

Il faut également définir un layout par type de lignes à afficher.
Si toutes les lignes ont un design identique, il ne faut définir qu'un seul layout.

Référence : https://developer.android.com/develop/ui/views/layout/recyclerview

Android UI Views : les RecyclerView

Le RecyclerView est le composant graphique permettant d'afficher une liste d'éléments.

Pour l'utiliser, il faut le coupler (par code) avec :

- Un RecyclerView.Adapter
  Un RecyclerView ne peut pas utiliser un ArrayAdapter (possible avec une ListView)
- Un RecyclerView.ViewHolder
- Un LayoutManager (LinearLayoutManager, GridLayoutManager ou StaggeredGridLayoutManager)

Référence : https://developer.android.com/develop/ui/views/layout/recyclerview?hl=fr

Android UI Views : RecyclerView.Adapter

Le RecyclerView.Adapter est la classe permettant d'alimenter les éléments à afficher dans la liste.
Deux méthodes doivent être obligatoirement redéfinies dans notre propre Adapter lors de l'héritage d'un :

- override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ItemViewHolder
  Permet la création des cellules de la liste
- override fun onBindViewHolder(holder: ItemViewHolder, position: Int)
  Permet de remplir les cellules de la liste
- override fun getItemCount() : Int
  Permet de définir le nombre d'éléments de la liste

Référence : https://developer.android.com/reference/androidx/recyclerview/widget/RecyclerView.Adapter

Android UI Views : RecyclerView.ViewHolder

Le RecyclerView.ViewHolder est la classe permettant de mémoriser les composants graphiques de chaque élément à afficher dans la liste.

Intérêt : Ne charger en mémoire que les composants graphiques visibles d'un élément pour pouvoir les réutiliser pour d'autres éléments à afficher.
On appelle cela le **recyclage de composants**.

Référence : https://developer.android.com/reference/androidx/recyclerview/widget/RecyclerView.ViewHolder

Android UI Views : LayoutManager

Le LayoutManager est la classe permettant d'assembler les cellules gérées par le RecyclerView.
On dispose de LayoutManager déjà existants :

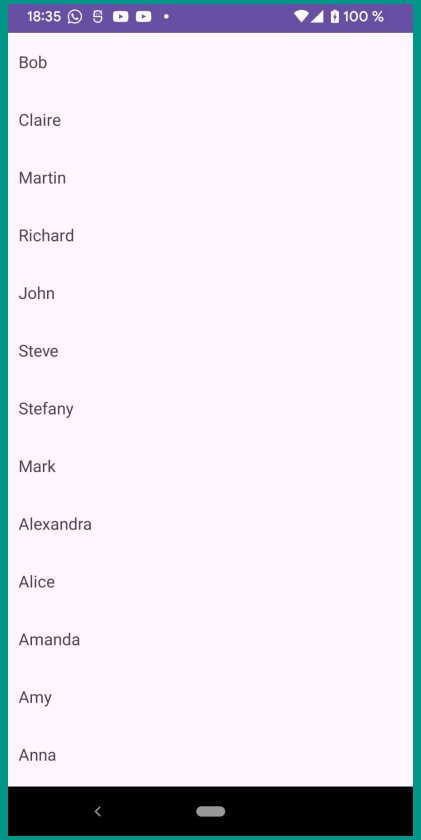
- GridLayoutManager
  Affichage en tableau/grille des éléments
- LinearLayoutManager
  Affichage en liste verticale ou horizontale des éléments
- StaggeredGridLayoutManager
  Affichage en tableau/grille des éléments mais en quinconce

Référence : https://developer.android.com/reference/androidx/recyclerview/widget/RecyclerView.LayoutManager

TD-ListSample

Exercice : Dans le projet Android “MyFirstAndroidApp”

1. Créez une nouvelle Activity “ListActivity” permettant d’afficher une liste d’éléments (RecyclerView)
2. Définissez une liste d’éléments de votre choix (des prénoms dans l’exemple ci-contre)
3. Implémentez un layout avec un RecyclerView
4. Implémentez le layout d’une ligne
5. Implémentez un **Adapter** à l’aide d’une classe qui va hériter de RecyclerView.Adapter<MyViewHolder> et codez les 3 méthodes indispensables :
   a. onCreateViewHolder(parent: ViewGroup, viewType: Int): MyViewHolder
   b. onBindViewHolder(holder: MyViewHolder, position: Int)
   c. getItemCount()
6. Implémentez un **ViewHolder** à l’aide d’une classe qui va hériter de RecyclerView.ViewHolder()
7. Dans l’Activity, utilisez le RecyclerView pour lui rattacher
   a. l’Adapter avec les éléments à afficher et
   b. un LayoutManager
8. Affichez les éléments dans la liste



Autres composants

Fenêtre Dialog

Les (fenêtres) Dialogs sont des fenêtres modales qui vont s'afficher sur l'écran (Activity courante).

Il existe plusieurs types de Dialogs prédéfinies :

- AlertDialog
- ProgressDialog (utilisation de ce composant déconseillée)
- DatePickerDialog
- TimePickerDialog

Référence : https://developer.android.com/develop/ui/views/components/dialogs

AlertDialog

Qu’est-ce ? (définition fonctionnelle)
Comme son nom l’indique, l’AlertDialog est une fenêtre de dialogue destinée à “alerter” l’utilisateur afin de l’avertir, de lui demander son aval... d’une action, d’un traitement.

Qu’est-ce ? (définition technique)
C’est une classe Java android.app.AlertDialog instanciée à l’aide de sa classe Builder

AlertDialog

Exemple d’implémentation :

```
AlertDialog.Builder builder = new AlertDialog.Builder(this);
builder.setMessage("Mon message").setTitle("Mon titre");
AlertDialog dialog = builder.create();
dialog.show();
```

Autres Dialogs

ProgressDialog
Fenêtre Dialog dédiée pour afficher une progression sur un traitement long.
Remarque : Il est conseillé d'utiliser plutôt une simple ProgressBar

DatePickerDialog et TimePickerDialog
Fenêtre Dialog dédiée pour afficher respectivement un composant complexe pour sélectionner une date ou une heure.

Toast et Snackbar

Toast et Snackbar sont des composants permettant d'afficher une information à l' écran de manière brève.

Le Toast permet d'afficher une information mais sans interaction possible de l'utilisateur.
Exemple: Toast.makeText(this, "Information", Toast.LENGTH_SHORT).show()

Le Snackbar est un composant plus "jeune" que le Toast et il offre, contrairement au Toast, la possibilité de valider un choix.
Exemple:

```
val aView = findViewById<ConstraintLayout>(R.id.main)
Snackbar.make(aView, "Information", Snackbar.LENGTH_SHORT).show()
```

Toast

Qu'est-ce ? (définition fonctionnelle)

Une mini fenêtre de message

Remarques : Il est possible de

- customiser le design d'un Toast (setView)
- modifier son positionnement à l'écran (setGravity)

Qu'est-ce ? (définition technique)

Une classe Toast qu'on paramètre et qu'on affiche

Référence : https://developer.android.com/reference/android/widget/Toast

Toast

Exemple d’implémentation :

```
val context = applicationContext
val text: CharSequence = "Hello toast!"
val duration = Toast.LENGTH_SHORT
val toast = Toast.makeText(context, text, duration)
toast.show()
```

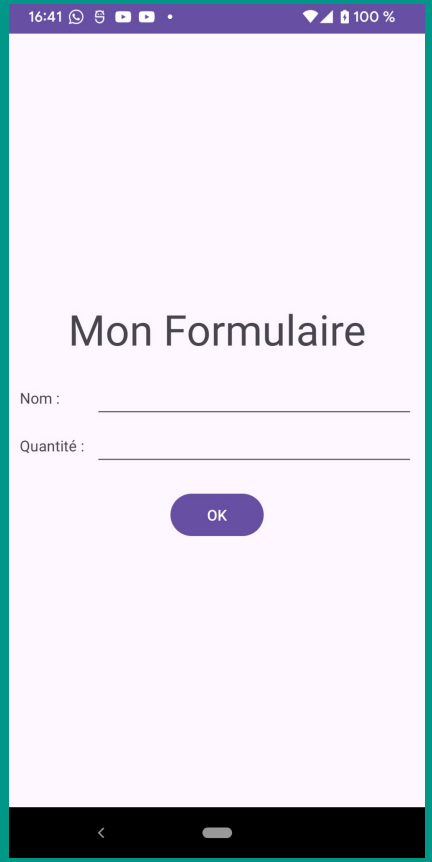
ou

```
Toast.makeText(applicationContext, "Mon message", Toast.LENGTH_SHORT).show()
```

TD-MyDesign (suite)

Exercice 1 :

1. Réutilisez le projet Android “MyDesign”
2. Gérez le click du bouton “OK”
   a. Si le “nom” et la “quantité” ne sont pas tous les deux renseignés, affichez un Toast avec un message l’indiquant



Menus

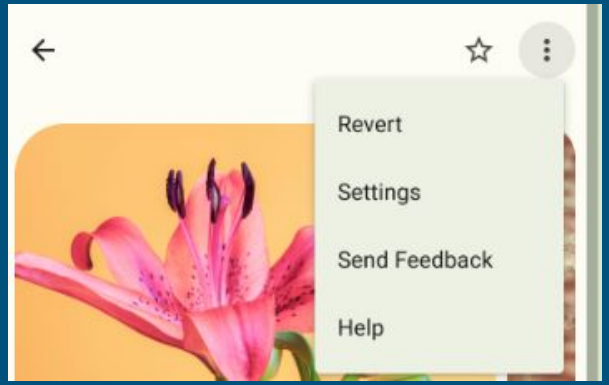
Les menus permettent de définir des actions qui pourront être déclenchées autrement que par de traditionnels boutons.

Il existe deux types de menus :

- Le **menu optionnel** : Il s'agit d'un menu défini par Activity
- Le **menu contextuel** : Il s'agit d'un menu défini sur un composant et généralement déclenché sur un appui long

Références :
https://developer.android.com/develop/ui/views/components/menus?hl=fr
https://developer.android.com/reference/android/view/Menu

Menus : Menu optionnel



Qu'est-ce ? (définition fonctionnelle)

Un menu optionnel est un menu qui peut s'afficher

- Dans la barre de menu de l'application, appellée Toolbar (anciennement ActionBar)
- lors d'un appui sur la touche "menu" du mobile (que sur les vieux appareils)

Il est défini par Activity.

Qu'est-ce ? (définition technique)

Les menus font partis des ressources de l'application et sont donc définis dans le répertoire "/res/menu/<nom du fichier>.xml"

Menus : Menu optionnel

Avantages

Il permet de définir des actions “optionnelles” disponibles sur l’Activity.
Il ne surcharge par le design. En effet, grâce à lui, inutile de mettre plusieurs boutons dans l’UI pour définir plusieurs actions, toutes peuvent être regroupées dans le menu.

Inconvénients

La mise en place d’un menu n’est pas forcément visible à l’utilisateur (pas d’indication visuelle) et donc pas intuitive.

Menus : Menu optionnel

Méthodes de l'Activity pour l'implémentation :

- onCreateOptionsMenu(Menu menu)
- onOptionsItemSelected(MenuItem item)
- onOptionsMenuClosed(Menu menu)

Menus : Menu contextuel

Qu’est-ce ? (définition fonctionnelle)
Un menu contextuel est un menu défini sur un élément graphique et qui s’affiche généralement sur un appui long sur le composant.

Il est le plus souvent défini sur une liste.

Qu’est-ce ? (définition technique)
Une fonctionnalité technique gérée sur une Activity pour des composants graphiques (View)

Référence : https://developer.android.com/reference/android/view/ContextMenu

Menus : Menu contextuel

Avantages

Il permet de définir des actions spécifiques sur un composant.
Il ne surcharge par le design. En effet, grâce à lui, inutile de rajouter des boutons d'actions au sein d'un composant, un menu contextuel va permettre cela.

Inconvénients

La présence ou non d'un menu contextuel sur un composant n'est pas visible et donc pas intuitive à l'utilisateur.

Pickers

Il existe plusieurs composants riches pour saisir une donnée (pickers) :

- DatePicker : Affiche un calendrier pour choisir une date
- TimePicker : Affiche une horloge pour choisir une heure
- ..

Références : https://developer.android.com/develop/ui/views/components/pickers
https://developer.android.com/reference/android/widget/DatePicker
https://developer.android.com/reference/android/widget/TimePicker

Questions/Réponses